Sec 13. CS Career

1. Security Principles

1 - CIA

- **Confidentiality** ensures that only the intended persons or recipients can access the data.
- **Integrity** aims to ensure that the data cannot be altered; moreover, we can detect any alteration if it occurs.
- **Availability** aims to ensure that the system or service is available when needed.

step beyond the CIA security triad, we can think of:

- **Authenticity**: Authentic means not fraudulent or counterfeit. Authenticity is about ensuring that the document/file/data is from the claimed source.
- **Nonrepudiation**: Repudiate means refusing to recognize the validity of something. Nonrepudiation ensures that the original source cannot deny that they are the source of a particular document/file/data. This characteristic is indispensable for various domains, such as shopping, patient diagnosis, and banking.

2 - DAD

The security of a system is attacked through one of several means. It can be via the disclosure of secret data, alteration of data, or destruction of data.

- **Disclosure** is the opposite of confidentiality. In other words, disclosure of confidential data would be an attack on confidentiality.
- **Alteration** is the opposite of Integrity. For example, the integrity of a cheque is indispensable.
- **Destruction/Denial** is the opposite of Availability.

The opposite of the CIA Triad would be the DAD Triad: Disclosure, Alteration, and Destruction. Protecting against disclosure, alteration, and destruction/denial is of utter significance. This protection is equivalent to working to maintain confidentiality, integrity and availability.

The attacker managed to gain access to customer records and dumped them online. What is this attack? Disclosure
A group of attackers were able to locate both the main and the backup power supply systems and switch them off. As a result, the whole network was shut down. What is this attack? Destruction/Denial

3 - Fundamental Concepts of Security Models

We have learned that the security triad is represented by Confidentiality, Integrity, and Availability (CIA). One might ask, how can we create a system that ensures one or more security functions? The answer would be in using security models. In this task, we will introduce three foundational security models:

- Bell-LaPadula Model
- The Biba Integrity Model
- The Clark-Wilson Model

Bell-LaPadula Model: The Bell-LaPadula Model aims to achieve **confidentiality** by specifying three rules:

- **Simple Security Property** : This property is referred to as “no read up”; it states that a subject at a lower security level cannot read an object at a higher security level. This rule prevents access to sensitive information above the authorized level.
- **Star Security Property** : This property is referred to as “no write down”; it states that a subject at a higher security level cannot write to an object at a lower security level. This rule prevents the disclosure of sensitive information to a subject of lower security level.
- **Discretionary-Security Property** : This property uses an access matrix to allow read and write operations. An example access matrix is shown in the table below and used in conjunction with the first two properties.

Biba Model: The Biba Model aims to achieve **integrity** by specifying two main rules:

- **Simple Integrity Property** : This property is referred to as “no read down”; a higher integrity subject should not read from a lower integrity object.
- **Star Integrity Property** : This property is referred to as “no write up”; a lower integrity subject should not write to a higher integrity object.

These two properties can be summarized as “read up, write down.” This rule is in contrast with the Bell-LaPadula Model, and this should not be surprising as one is concerned with confidentiality while the other is with integrity.

Biba Model suffers from various limitations. One example is that it does not handle internal threats (insider threat).

Clark-Wilson Model: The Clark-Wilson Model also aims to achieve integrity by using the following concepts:

- **Constrained Data Item (CDI)** : This refers to the data type whose integrity we want to preserve.
- **Unconstrained Data Item (UDI)** : This refers to all data types beyond CDI, such as user and system input.
- **Transformation Procedures (TPs)** : These procedures are programmed operations, such as read and write, and should maintain the integrity of CDIs.
- **Integrity Verification Procedures (IVPs)** : These procedures check and ensure the validity of CDIs.

We covered only three security models. The reader can explore many additional security models. Examples include:

- Brewer and Nash model
- Goguen-Meseguer model
- Sutherland model
- Graham-Denning model
- Harrison-Ruzzo-Ullman model

4 - Defence-in-Depth

Defence-in-Depth refers to creating a security system of multiple levels; hence it is also called Multi-Level Security. Consider the following analogy: you have a locked drawer where you keep your important documents and pricey stuff. The drawer is locked; however, do you want this drawer lock to be the only thing standing between a thief and your expensive items? If we think of multi-level security, we would prefer that the drawer be locked, the relevant room be locked, the main door of the apartment be locked, the building gate be locked, and you might even want to throw in a few security cameras along the way. Although these multiple levels of security cannot stop every thief, they would block most of them and slow down the others.

5 - ISO/IEC 19249

The International Organization for Standardization (ISO) and the International Electrotechnical Commission (IEC) have created the ISO/IEC 19249. In this task, we will brush briefly upon ISO/IEC 19249:2017 *Information technology - Security techniques - Catalogue of architectural and design principles for secure products, systems and applications*. The purpose is to have a better idea of what international organizations would teach regarding security principles.

ISO/IEC 19249 lists five *architectural* principles:

1. **Domain Separation**: Every set of related components is grouped as a single entity; components can be applications, data, or other resources. Each entity will have its own domain and be assigned a common set of security attributes. For example, consider the x86 processor privilege levels: the operating system kernel can run in ring 0 (the most privileged level). In contrast, user-mode applications can run in ring 3 (the least privileged level). Domain separation is included in the Goguen-Meseguer Model.
2. **Layering**: When a system is structured into many abstract levels or layers, it becomes possible to impose security policies at different levels; moreover, it would be feasible to validate the operation. Let’s consider the OSI (Open Systems Interconnection) model with its seven layers in networking. Each layer in the OSI model provides specific services to the layer above it. This layering makes it possible to impose security policies and easily validate that the system is working as intended. Another example from the programming world is disk operations; a programmer usually uses the disk read and write

functions provided by the chosen high-level programming language. The programming language hides the low-level system calls and presents them as more user-friendly methods. Layering relates to Defence in Depth.

3. **Encapsulation**: In object-oriented programming (OOP), we hide low-level implementations and prevent direct manipulation of the data in an object by providing specific methods for that purpose. For example, if you have a clock object, you would provide a method `increment()` instead of giving the user direct access to the `seconds` variable. The aim is to prevent invalid values for your variables. Similarly, in larger systems, you would use (or even design) a proper Application Programming Interface (API) that your application would use to access the database.
4. **Redundancy**: This principle ensures availability and integrity. There are many examples related to redundancy. Consider the case of a hardware server with two built-in power supplies: if one power supply fails, the system continues to function. Consider a RAID 5 configuration with three drives: if one drive fails, data remains available using the remaining two drives. Moreover, if data is improperly changed on one of the disks, it would be detected via the parity, ensuring the data's integrity.
5. **Virtualization**: With the advent of cloud services, virtualization has become more common and popular. The concept of virtualization is sharing a single set of hardware among multiple operating systems. Virtualization provides sandboxing capabilities that improve security boundaries, secure detonation, and observance of malicious programs.

ISO/IEC 19249 teaches five *design* principles:

1. **Least Privilege**: You can also phrase it informally as "need-to basis" or "need-to-know basis" as you answer the question, "who can access what?" The principle of least privilege teaches that you should provide the least amount of permissions for someone to carry out their task and nothing more. For example, if a user needs to be able to view a document, you should give them read rights without write rights.
2. **Attack Surface Minimisation**: Every system has vulnerabilities that an attacker might use to compromise a system. Some vulnerabilities are known, while others are yet to be discovered. These vulnerabilities represent risks that we

should aim to minimize. For example, in one of the steps to harden a Linux system, we would disable any service we don't need.

3. **Centralized Parameter Validation**: Many threats are due to the system receiving input, especially from users. Invalid inputs can be used to exploit vulnerabilities in the system, such as denial of service and remote code execution. Therefore, parameter validation is a necessary step to ensure the correct system state. Considering the number of parameters a system handles, the validation of the parameters should be centralized within one library or system.
4. **Centralized General Security Services**: As a security principle, we should aim to centralize all security services. For example, we would create a centralized server for authentication. Of course, you might take proper measures to ensure availability and prevent creating a single point of failure.
5. **Preparing for Error and Exception Handling**: Whenever we build a system, we should take into account that errors and exceptions do and will occur. For instance, in a shopping application, a customer might try to place an order for an out-of-stock item. A database might get overloaded and stop responding to a web application. This principle teaches that the systems should be designed to fail safe; for example, if a firewall crashes, it should block all traffic instead of allowing all traffic. Moreover, we should be careful that error messages don't leak information that we consider confidential, such as dumping memory content that contains information related to other customers.

In the following questions, refer to the ISO/IEC 19249 five design principles above. Answer with a number between 1 and 5, depending on the number of the design principle.

Which principle are you applying when you turn off an insecure server that is not critical to the business? 2
Your company hired a new sales representative. Which principle are they applying when they tell you to give them access only to the company products and prices? 1
While reading the code of an ATM, you noticed a huge chunk of code to handle unexpected situations such as network disconnection and power failure. Which principle are they applying? 5

6- Zero Trust versus Trust but Verify

Trust is a very complex topic; in reality, we cannot function without trust. If one were to think that the laptop vendor has installed spyware on the laptop, they would most likely end up rebuilding the system. If one were to mistrust the hardware vendor, they would stop using it completely. If we think of trust on a business level, things only become more sophisticated; however, we need some guiding security principles. Two security principles that are of interest to us regarding trust:

- Trust but Verify
- Zero Trust

Trust but Verify: This principle teaches that we should always verify even when we trust an entity and its behaviour. An entity might be a user or a system. Verifying usually requires setting up proper logging mechanisms; verifying indicates going through the logs to ensure everything is normal. In reality, it is not feasible to verify everything; just think of the work it takes to review all the actions taken by a single entity, such as Internet pages browsed by a single user. This requires automated security mechanisms, such as proxy, intrusion detection, and intrusion prevention systems.

Zero Trust: This principle treats trust as a vulnerability, and consequently, it caters to insider-related threats. After considering trust as a vulnerability, zero trust tries to eliminate it. It is teaching indirectly, “never trust, always verify.” In other words, every entity is considered adversarial until proven otherwise. Zero trust does not grant trust to a device based on its location or ownership. This approach contrasts with older models that would trust internal networks or enterprise-owned devices. Authentication and authorization are required before accessing any resource. As a result, if any breach occurs, the damage would be more contained if a zero trust architecture had been implemented.

Microsegmentation is one of the implementations used for Zero Trust. It refers to the design where a network segment can be as small as a single host. Moreover, communication between segments requires authentication, access control list checks, and other security requirements.

There is a limit to how much we can apply zero trust without negatively impacting a business; however, this does not mean that we should not apply it as long as it is feasible

7- Threat versus Risk

There are three terms that we need to take note of to avoid any confusion.

- **Vulnerability**: Vulnerable means susceptible to attack or damage. In information security, a vulnerability is a weakness.
- **Threat**: A threat is a potential danger associated with this weakness or vulnerability.
- **Risk**: The risk is concerned with the likelihood of a threat actor exploiting a vulnerability and the consequent impact on the business.

Away from information systems, a showroom with doors and windows made of standard glass suffers a weakness, or *vulnerability*, due to the nature of glass. Consequently, there is a *threat* that the glass doors and windows can be broken. The showroom owners should contemplate the *risk*, i.e. the likelihood that a glass door or window gets broken and the resulting impact on the business.

Consider another example directly related to information systems. You work for a hospital that uses a particular database system to store all the medical records. One day, you are following the latest security news, and you learn that the used database system is not only vulnerable but also a proof-of-concept working exploit code has been released; the released exploit code indicates that the threat is real. With this knowledge, you must consider the resulting risk and decide the next steps.

2. Careers in Cyber

1. Security Analyst

Security analysts are integral to constructing security measures across organisations to protect the company from attacks. Analysts explore and evaluate company networks to uncover actionable data and recommendations for

engineers to develop preventative measures. This job role requires working with various stakeholders to gain an understanding of security requirements and the security landscape. **Responsibilities**

- Working with various stakeholders to analyse the cyber security throughout the company
- Compile ongoing reports about the safety of networks, documenting security issues and measures taken in response
- Develop security plans, incorporating research on new attack tools and trends, and measures needed across teams to maintain data security.

2. Security Engineer

Security engineers develop and implement security solutions using threats and vulnerability data - often sourced from members of the security workforce. Security engineers work across circumventing a breadth of attacks, including web application attacks, network threats, and evolving trends and tactics. The ultimate goal is to retain and adopt security measures to mitigate the risk of attack and data loss. **Responsibilities**

- Testing and screening security measures across software
- Monitor networks and reports to update systems and mitigate vulnerabilities
- Identify and implement systems needed for optimal security

3. Incident Responder

Incident responders respond productively and efficiently to security breaches. Responsibilities include creating plans, policies, and protocols for organisations to enact during and following incidents. This is often a highly pressurised position with assessments and responses required in real-time, as attacks are unfolding. Incident response metrics include MTTD, MTTA, and MTTR - the meantime to detect, acknowledge, and recover (from attacks.) The aim is to achieve a swift and effective response, retain financial standing and avoid negative breach implications. Ultimately, incident responders protect the company's data, reputation, and financial standing from cyber attacks. **Responsibilities**

- Developing and adopting a thorough, actionable incident response plan
- Maintaining strong security best practices and supporting incident response measures
- Post-incident reporting and preparation for future attacks, considering learnings and adaptations to take from incidents

4 . Digital Forensics Examiner

If you like to play detective, this might be the perfect job. If you are working as part of a law-enforcement department, you would be focused on collecting and analysing evidence to help solve crimes: charging the guilty and exonerating the innocent. On the other hand, if your work falls under defending a company's network, you will be using your forensic skills to analyse incidents, such as policy violations. **Responsibilities**

- Collect digital evidence while observing legal procedures
- Analyse digital evidence to find answers related to the case
- Document your findings and report on the case

5. Malware Analyst

A malware analyst's work involves analysing suspicious programs, discovering what they do and writing reports about their findings. A malware analyst is sometimes called a reverse-engineer as their core task revolves around converting compiled programs from machine language to readable code, usually in a low-level language. This work requires the malware analyst to have a strong programming background, especially in low-level languages such as assembly language and C language. The ultimate goal is to learn about all the activities that a malicious program carries out, find out how to detect it and report it.
Responsibilities

- Carry out static analysis of malicious programs, which entails reverse-engineering
- Conduct dynamic analysis of malware samples by observing their activities in a controlled environment

- Document and report all the findings

6. Penetration Tester

You may see penetration testing referred to as pentesting and ethical hacking. A penetration tester's job role is to test the security of the systems and software within a company - this is achieved through attempts to uncover flaws and vulnerabilities through systemised hacking. Penetration testers exploit these vulnerabilities to evaluate the risk in each instance. The company can then take these insights to rectify issues to prevent a real-world cyberattack.
Responsibilities

- Conduct tests on computer systems, networks, and web-based applications
- Perform security assessments, audits, and analyse policies
- Evaluate and report on insights, recommending actions for attack prevention

7. Red Teamer

Red teamers share similarities to penetration testers, with a more targeted job role. Penetration testers look to uncover many vulnerabilities across systems to keep cyber-defence in good standing, whilst red teamers are enacted to test the company's detection and response capabilities. This job role requires imitating cyber criminals' actions, emulating malicious attacks, retaining access, and avoiding detection. Red team assessments can run for up to a month, typically by a team external to the company. They are often best suited to organisations with mature security programs in place. **Responsibilities**

- Emulate the role of a threat actor to uncover exploitable vulnerabilities, maintain access and avoid detection
- Assess organisations' security controls, threat intelligence, and incident response procedures
- Evaluate and report on insights, with actionable data for companies to avoid real-world instances